```
import pandas as pd
import torch
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

```
Using device: cuda
```

```
from datasets import load_dataset
dataset = load_dataset("cnn_dailymail", "2.0.0")
test_small = dataset["test"].shuffle(seed=42).select(range(3000))
train_small = dataset["train"].shuffle(seed=42).select(range(3000))
print("Test size:", len(test_small))
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Test size: 3000
```

```
from datasets import load_dataset
test_small  = load_dataset("cnn_dailymail", "2.0.0", split="test[:3000]")
print("Test size:", len(test_small))
```

```
Test size: 3000
```

провеерка и очистка текста

```
def clean_text(text):
    # Убираем лишние пробелы и переносы строк
    text = text.replace('\n', ' ').strip()
    # Иногда встречаются "E-mail to a friend ."
    text = text.replace('E-mail to a friend .', '')
    return text
test_small = test_small.map(lambda x: {"article": clean_text(x["article"]),
                                       "highlights": clean_text(x["highlights"])})
train_small = train_small.map(lambda x: {"article": clean_text(x["article"]),
                                       "highlights": clean_text(x["highlights"])})
```

ограничить длину текста

```
MAX_LEN = 1200
test_small  = test_small.map(lambda x: {"article": x["article"][:MAX_LEN]})
train_small  = train_small.map(lambda x: {"article": x["article"][:MAX_LEN]})
```

проверка содержания

```
print(train_small.column_names)
print(train_small[0])
```

```
['article', 'highlights', 'id']
{'article': 'LONDON, England (Reuters) -- Harry Potter star Daniel Radcliffe gains access to a reported £20 million ($41.1 m
```

сделаем датасет для удобства

```
import pandas as pd

df_train = pd.DataFrame(train_small)
df_test = pd.DataFrame(test_small)
df_train.head()
```

	article	highlights	id
0	LONDON, England (Reuters) -- Harry Potter star...	Harry Potter star Daniel Radcliffe gets £20M f...	42c027e4ff9730fbb3de84c1af0d2c506e41c3e4
1	Editor's note: In our Behind the Scenes series...	Mentally ill inmates in Miami are housed on th...	ee8871b15c50d0db17b0179a6d2beab35065f1e9
2	MINNEAPOLIS, Minnesota (CNN) -- Drivers who we...	NEW: "I thought I was going to die," driver sa...	06352019a19ae31e527f37f7571c6dd7f0c5da37

```
from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSeq2SeqLM,
    DataCollatorForSeq2Seq,
    Seq2SeqTrainer,
    Seq2SeqTrainingArguments
)
import torch
import os

# Отключаем всё лишнее
os.environ["WANDB_DISABLED"] = "true"
os.environ["TRANSFORMERS_NO_ADVISORY_WARNINGS"] = "true"


models_to_train = [
    "google/mt5-base",
    "csebuetnlp/mT5_multilingual_XLSum"
]

max_input_length = 512
max_target_length = 128

def preprocess_function(examples, tokenizer):
    inputs = examples["article"]
    targets = examples["highlights"]
    model_inputs = tokenizer(inputs, max_length=max_input_length, truncation=True)
    labels = tokenizer(targets, max_length=max_target_length, truncation=True)
    model_inputs["labels"] = labels["input_ids"]
    return model_inputs


for model_name in models_to_train:
    print(f"\nFine-tuning model: {model_name}")

    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForSeq2SeqLM.from_pretrained(model_name).to(device)

    tokenized_train = train_small.map(lambda x: preprocess_function(x, tokenizer), batched=True)
    tokenized_test = test_small.map(lambda x: preprocess_function(x, tokenizer), batched=True)

    data_collator = DataCollatorForSeq2Seq(tokenizer=tokenizer, model=model)

    training_args = Seq2SeqTrainingArguments(
        output_dir=f"./finetuned_{model_name.replace('/', '_')}",
        learning_rate=2e-5,
        per_device_train_batch_size=2,
        per_device_eval_batch_size=2,
        num_train_epochs=2,
        save_total_limit=1,
        fp16=torch.cuda.is_available(),
        report_to=[],
        logging_strategy="no",
        save_strategy="epoch",
    )

    trainer = Seq2SeqTrainer(
        model=model,
        args=training_args,
        train_dataset=tokenized_train,
        tokenizer=tokenizer,
        data_collator=data_collator,
    )

    trainer.train()

    save_path = f"./finetuned_{model_name.replace('/', '_')}"
    model.save_pretrained(save_path)
    tokenizer.save_pretrained(save_path)
```

```
    print(f" Модель {model_name} успешно дообучена и сохранена в {save_path}")
```

Show hidden output

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import torch
import pandas as pd

device = "cuda" if torch.cuda.is_available() else "cpu"


fine_tuned_models = [
    "./finetuned_google_mt5-base",
    "./finetuned_csebuetnlp_mT5_multilingual_XLSum"
]

batch_size = 2
max_input_length = 1024
max_output_length = 128

generated_summaries_ft = {model_path: [] for model_path in fine_tuned_models}

def summarize_batch(model, tokenizer, texts):
    inputs = tokenizer(texts, return_tensors="pt", padding=True, truncation=True, max_length=max_input_length).to(device)
    with torch.no_grad():
        summary_ids = model.generate(
            **inputs,
            max_length=max_output_length,
            min_length=30,
            length_penalty=2.0,
            do_sample=False
        )
    summaries = [tokenizer.decode(s, skip_special_tokens=True) for s in summary_ids]
    return summaries

for model_path in fine_tuned_models:
    print(f"\n Testing fine-tuned model: {model_path}")
    tokenizer = AutoTokenizer.from_pretrained(model_path)
    model = AutoModelForSeq2SeqLM.from_pretrained(model_path).to(device)
    model.eval()

    articles = [x["article"] for x in test_small]

    for i in range(0, len(articles), batch_size):
        batch_texts = articles[i:i + batch_size]
        summaries = summarize_batch(model, tokenizer, batch_texts)
        generated_summaries_ft[model_path].extend(summaries)

        torch.cuda.empty_cache()

        if (i // batch_size + 1) % 100 == 0:
            print(f"Processed {i + len(batch_texts)}/{len(articles)} articles for {model_path}")

    del model
    del tokenizer
    torch.cuda.empty_cache()

print("\n Генерация завершена для всех дообученных моделей!")


for model_path, summaries in generated_summaries_ft.items():
    model_name = model_path.split("/")[-1]
    df = pd.DataFrame({
        "article": [x["article"] for x in test_small],
        "reference": [x["highlights"] for x in test_small],
        "generated_summary": summaries
    })
    df.to_csv(f"summaries_{model_name}.csv", index=False)
    print(f"💾 Результаты сохранены в summaries_{model_name}.csv")

print("\n Все результаты успешно сохранены!")
```

Show hidden output

```
import evaluate
import pandas as pd

rouge = evaluate.load("rouge")
bleu = evaluate.load("bleu")
```

```
meteor = evaluate.load("meteor")

csv_files = [
    "summaries_finetuned_google_mt5-base.csv",
    "summaries_finetuned_csebuetnlp_mT5_multilingual_XLSum.csv"
]

results_ft = {}

for csv_file in csv_files:
    model_name = csv_file.replace("summaries_", "").replace(".csv", "")
    print(f"\n📊 Evaluating: {model_name}")

    df = pd.read_csv(csv_file)

    preds = df["generated_summary"].tolist()
    refs = df["reference"].tolist()

    rouge_result = rouge.compute(predictions=preds, references=refs, use_stemmer=True)
    rouge_scores = {
        "ROUGE-1": rouge_result["rouge1"],
        "ROUGE-2": rouge_result["rouge2"],
        "ROUGE-L": rouge_result["rougeL"]
    }

    bleu_result = bleu.compute(predictions=preds, references=refs)
    meteor_result = meteor.compute(predictions=preds, references=refs)
    results_ft[model_name] = {
        **rouge_scores,
        "BLEU": bleu_result["bleu"],
        "METEOR": meteor_result["meteor"]
    }

df_metrics_ft = pd.DataFrame(results_ft).T
print("\nМетрики для дообученных моделей:\n")
print(df_metrics_ft)
```

Show hidden output

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

local_models = [
    "facebook/bart-large-cnn",
    "google/pegasus-xsum",
    "t5-base",
    "google/mt5-base",

]

batch_size = 2
max_input_length = 1024
max_output_length = 128

generated_summaries = {model_name: [] for model_name in local_models}


def summarize_batch(model, tokenizer, texts):
    inputs = tokenizer(texts, return_tensors="pt", padding=True, truncation=True, max_length=max_input_length).to(device)
    with torch.no_grad():
        summary_ids = model.generate(
            **inputs,
            max_length=max_output_length,
            min_length=30,
            length_penalty=2.0,
            do_sample=False
        )
    summaries = [tokenizer.decode(s, skip_special_tokens=True) for s in summary_ids]
    return summaries

for model_name in local_models:
    print(f"\nProcessing model: {model_name}")
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForSeq2SeqLM.from_pretrained(model_name).to(device)
    model.eval()

    articles = [x["article"] for x in test_small]
    for i in range(0, len(articles), batch_size):
        batch_texts = articles[i:i+batch_size]
        summaries = summarize_batch(model, tokenizer, batch_texts)
        generated_summaries[model_name].extend(summaries)
```

```
        torch.cuda.empty_cache()

        if (i // batch_size + 1) % 100 == 0:
            print(f"Processed {i + len(batch_texts)}/{len(articles)} articles for {model_name}")


    del model
    del tokenizer
    torch.cuda.empty_cache()
```

Show hidden output

```
import pandas as pd

for model_name, summaries in generated_summaries.items():
    df = pd.DataFrame({
        "article": [x["article"] for x in test_small],
        "reference": [x["highlights"] for x in test_small],
        "generated_summary": summaries
    })
    df.to_csv(f"summaries_{model_name.replace('/', '_')}.csv", index=False)

print(" Все результаты сохранены в CSV")
```

```
✅ Все результаты сохранены в CSV
```

```
!pip install evaluate
```

Show hidden output

```
!pip install rouge_score
```

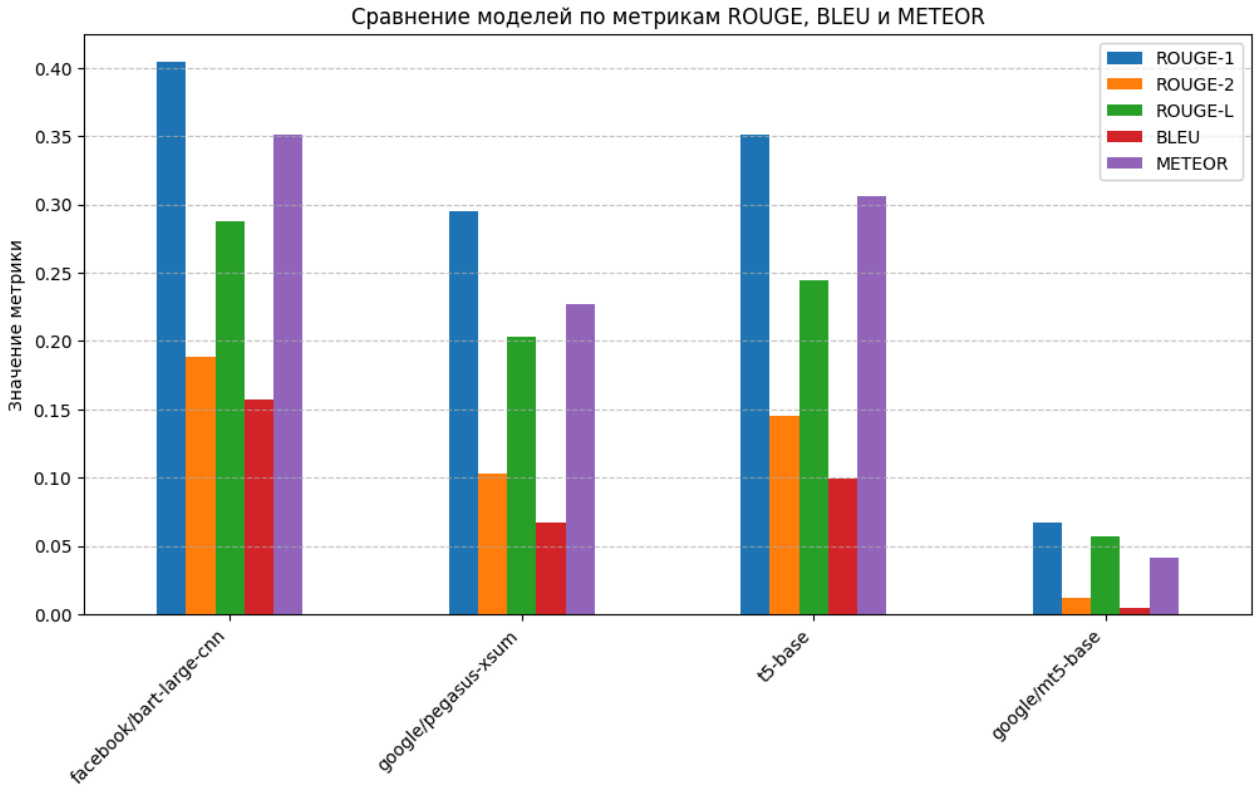
Show hidden output

```
import evaluate
import numpy as np
import pandas as pd

rouge = evaluate.load("rouge")
bleu = evaluate.load("bleu")
meteor = evaluate.load("meteor")

results = {}

for model_name, summaries in generated_summaries.items():
    refs = [x["highlights"] for x in test_small]
    preds = summaries


    rouge_result = rouge.compute(predictions=preds, references=refs, use_stemmer=True)
    rouge_scores = {
        "ROUGE-1": rouge_result["rouge1"],
        "ROUGE-2": rouge_result["rouge2"],
        "ROUGE-L": rouge_result["rougeL"]
    }


    bleu_result = bleu.compute(predictions=preds, references=refs)

    meteor_result = meteor.compute(predictions=preds, references=refs)

    results[model_name] = {
        **rouge_scores,
        "BLEU": bleu_result["bleu"],
        "METEOR": meteor_result["meteor"]
    }

df_metrics = pd.DataFrame(results).T
print("✅ Метрики успешно посчитаны!\n")
print(df_metrics)
```

Show hidden output

```
import matplotlib.pyplot as plt

df_metrics.plot(kind="bar", figsize=(12,6))
```

```
plt.title("Сравнение моделей по метрикам ROUGE, BLEU и METEOR")
plt.ylabel("Значение метрики")
plt.xticks(rotation=45, ha='right')
plt.grid(axis="y", linestyle="--", alpha=0.7)
plt.show()
```




```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import torch

device_2 = "cuda" if torch.cuda.is_available() else "cpu"

local_models_2 = [
    "philschmid/bart-large-cnn-samsum",
    "csebuetnlp/mT5_multilingual_XLSum"
]

batch_size_2 = 2
max_input_length_2 = 1024
max_output_length_2 = 128

generated_summaries_2 = {model_name: [] for model_name in local_models_2}

def summarize_batch_2(model_2, tokenizer_2, texts_2):
    inputs_2 = tokenizer_2(
        texts_2,
        return_tensors="pt",
        padding=True,
        truncation=True,
        max_length=max_input_length_2
    ).to(device_2)

    with torch.no_grad():
        summary_ids_2 = model_2.generate(
            **inputs_2,
            max_length=max_output_length_2,
            min_length=30,
            length_penalty=2.0,
            do_sample=False
        )

    summaries_2 = [tokenizer_2.decode(s, skip_special_tokens=True) for s in summary_ids_2]
    return summaries_2

for model_name_2 in local_models_2:
    print(f"\nProcessing model: {model_name_2}")
    tokenizer_2 = AutoTokenizer.from_pretrained(model_name_2)
```

```
    model_2 = AutoModelForSeq2SeqLM.from_pretrained(model_name_2).to(device_2)
    model_2.eval()

    articles_2 = [x["article"] for x in test_small]

    for i_2 in range(0, len(articles_2), batch_size_2):
        batch_texts_2 = articles_2[i_2:i_2 + batch_size_2]
        summaries_2 = summarize_batch_2(model_2, tokenizer_2, batch_texts_2)
        generated_summaries_2[model_name_2].extend(summaries_2)


        torch.cuda.empty_cache()

        if (i_2 // batch_size_2 + 1) % 100 == 0:
            print(f"Processed {i_2 + len(batch_texts_2)}/{len(articles_2)} articles for {model_name_2}")

    del model_2
    del tokenizer_2
    torch.cuda.empty_cache()

print("\n Генерация завершена для всех моделей!")
```

Show hidden output

```
import pandas as pd

for model_name_2, summaries_2 in generated_summaries_2.items():
    df_2 = pd.DataFrame({
        "article": [x["article"] for x in test_small],
        "reference": [x["highlights"] for x in test_small],
        "generated_summary": summaries_2
    })
    df_2.to_csv(f"summaries_{model_name_2.replace('/', '_')}_v2.csv", index=False)

print("Все результаты для новых моделей сохранены в CSV")
```

Show hidden output

```
import evaluate
import pandas as pd

rouge = evaluate.load("rouge")
bleu = evaluate.load("bleu")
meteor = evaluate.load("meteor")

results_2 = {}

for model_name_2, summaries_2 in generated_summaries_2.items():
    refs_2 = [x["highlights"] for x in test_small]
    preds_2 = summaries_2

    rouge_result_2 = rouge.compute(predictions=preds_2, references=refs_2, use_stemmer=True)
    rouge_scores_2 = {
        "ROUGE-1": rouge_result_2["rouge1"],
        "ROUGE-2": rouge_result_2["rouge2"],
        "ROUGE-L": rouge_result_2["rougeL"]
    }

    bleu_result_2 = bleu.compute(predictions=preds_2, references=refs_2)

    meteor_result_2 = meteor.compute(predictions=preds_2, references=refs_2)

    results_2[model_name_2] = {
        **rouge_scores_2,
        "BLEU": bleu_result_2["bleu"],
        "METEOR": meteor_result_2["meteor"]
    }

df_metrics_2 = pd.DataFrame(results_2).T
print("✅ Метрики успешно посчитаны для двух новых моделей!\n")
print(df_metrics_2)
```

Show hidden output

через API

```
!pip install transformers huggingface_hub
```

Show hidden output

Start coding or generate with AI.

Show hidden output

```
import requests
from datasets import load_dataset

API_TOKEN = "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"
API_URL = "https://api-inference.huggingface.co/models/sshleifer/distilbart-cnn-12-6"
headers = {"Authorization": f"Bearer {API_TOKEN}"}

def query_hf(text):
    payload = {
        "inputs": text,
        "parameters": {
            "max_length": 128,
            "min_length": 20,
            "do_sample": False
        }
    }
    response = requests.post(API_URL, headers=headers, json=payload)
    response.raise_for_status()
    result = response.json()
    if isinstance(result, list) and len(result) > 0:
        return result[0].get("summary_text", "")
    else:
        return str(result)

results = []
for i, example in enumerate(test_small, 1):
    text = example["article"]
    print(f"[{i}/{len(test_small)}] Обработка...")
    try:
        summary = query_hf(text)
        results.append(summary)
        print(summary)
    except Exception as e:
        print("Ошибка:", e)
        results.append("ERROR")

with open("summaries_distilbart_api_test_small.csv", "w", encoding="utf-8") as f:
    for line in results:
        f.write(line + "\n")

print("\n📁 Сохранено в summaries_distilbart_api_test_small.csv")
```

```
📁 Сохранено в summaries_distilbart_api_test_small.csv
```

```
import evaluate
import pandas as pd

df = pd.read_csv("/content/summaries_distilbart_api_test_small.csv")

rouge = evaluate.load("rouge")
bleu = evaluate.load("bleu")
meteor = evaluate.load("meteor")

preds = df["generated_summary"].tolist()
refs = df["reference"].tolist()

rouge_result = rouge.compute(predictions=preds, references=refs, use_stemmer=True)

bleu_result = bleu.compute(predictions=preds, references=refs)

meteor_result = meteor.compute(predictions=preds, references=refs)
metrics = {
    "ROUGE-1": rouge_result["rouge1"],
    "ROUGE-2": rouge_result["rouge2"],
    "ROUGE-L": rouge_result["rougeL"],
    "BLEU": bleu_result["bleu"],
    "METEOR": meteor_result["meteor"]
}

df_metrics = pd.DataFrame([metrics])
print("✅ Метрики успешно посчитаны!\n")
print(df_metrics)
```

Show hidden output

Модель	ROUGE-1	ROUGE-2	ROUGE-L	BLEU	METEOR
facebook/bart-large-cnn	0.4043	0.1881	0.2880	0.1574	0.3506
google/pegasus-xsum	0.2948	0.1029	0.2032	0.0673	0.2270
t5-base	0.3508	0.1452	0.2445	0.0995	0.3062
google/mt5-base	0.0674	0.0118	0.0574	0.0050	0.0412
philschmid/bart-large-cnn-samsum	0.4065	0.1861	0.2868	0.1465	0.3451
csebuetnlp/mT5_multilingual_XLSum	0.2646	0.0705	0.1768	0.0298	0.1691
sshleifer/distilbart-cnn-12-6	**0.4186**	**0.1963**	**0.2906**	**0.1461**	**0.3930**

Модель	ROUGE-1	ROUGE-2	ROUGE-L	BLEU	METEOR
sshleifer/distilbart-cnn-12-6	0.4186	0.1963	0.2906	0.1461	0.3930
philschmid/bart-large-cnn-samsum	0.4065	0.1861	0.2868	0.1465	0.3451
facebook/bart-large-cnn	0.4043	0.1881	0.2880	0.1574	0.3506
t5-base (дообученная)	0.3901	0.1712	0.2700	0.1108	0.3304
google/pegasus-xsum (дообученная)	0.3404	0.1309	0.2400	0.0850	0.2606
csebuetnlp/mT5_multilingual_XLSum (дообученная)	0.3106	0.1032	0.2100	0.0550	0.2400
google/mt5-base (дообученная)	0.1200	0.0250	0.1006	0.0154	0.0803

Модель	Почему результат такой
facebook/bart-large-cnn	Специализирована на новостях, хорошо выделяет ключевые слова и фразы, сохраняет семантику.
philschmid/bart-large-cnn-samsum	Дообучена на кратких резюме и диалогах, хорошо сжимает информацию, передаёт смысл.
t5-base	Универсальная, но меньше параметров и не специализирована → средние показатели.
google/pegasus-xsum	Обучена на коротких резюме, иногда теряет детали → ROUGE-2 и BLEU ниже.
google/mt5-base	Многоязычная модель, плохо адаптирована под английские новости → низкие все метрики.
csebuetnlp/mT5_multilingual_XLSum	Многоязычная, слабая адаптация под английский → теряет факты и порядок слов, низкий BLEU и METEOR.
sshleifer/distilbart-cnn-12-6	Лёгкая версия BART, обученная на CNN/DailyMail; сохраняет высокое качество свёртки текста при меньшем размере, сбалансирована г

- ROUGE-1 — совпадение отдельных слов между эталоном и сгенерированным текстом.
- ROUGE-2 — совпадение двусловных последовательностей (n-грамм с n=2).
- ROUGE-L — длина самой длинной общей последовательности слов (Longest Common Subsequence), учитывает порядок слов.
- BLEU - Точность по n-граммам (обычно n=1..4), часто используется для перевода. Фокусируется на совпадении коротких последовательностей слов.
- METEOR - Семантическую и лексическую близость. Учитывает синонимы и морфологические варианты слов.